Dynamic Analysis Test App - 1.0 - Android - SAST Scan Report

Q

74 /100 *
Security Rating

Package Name : com.quixxi.dynamic.testapp

Report generated on 20 Aug 2025 04:37 AM

This document summarizes the findings, analysis and recommendations from the assessment conducted by Quixxi

CONFIDENTIALITY AND LIABILITY

The contents of this document are intended solely for "Dynamic Analysis Test App", may contain confidential and/or privileged information and may be legally protected from disclosure. Without the prior consent of "Dynamic Analysis Test App" this document must not be printed, photocopied, given to any third party or shared in electronic form such as email, in whole or in part. If you are not the intended recipient of this document - or if this has been addressed to you in error - please alert the sender immediately.

DISCLAIMER

Quixxi will not be responsible for any reputational and/or revenue loss due to the testing. In particular:

- Please note that the information presented in this report along with the recommendations are the final delivery items from Quixxi scan. Quixxi does not warrant any further action nor does it promise remedial action. By using the scan service, the user understands that Quixxi does not provide any consultation, service or promise to remedy any vulnerabilities identified by Quixxi scan and that it is the user's responsibility to choose to take further action.
- * Security rating is calculated based on the number and severity of vulnerabilities detected. Higher rating means better security. Regardless of the score, reviewing the full report details is recommended for a complete understanding.
- This vulnerability assessment is a static analysis of your app. For this reason Quixxi will not try to run your app and - above all - will not try to hack it. It must be intended as a review of the code underlying the app itself.
- By its nature the test scans only the vulnerabilities listed in the reports and only for the app version provided.
- The test is meant to find possible vulnerabilities based on the app provided by customer. An incorrect app provision can result in wrong detections. Quixxi will not be liable for such situations.
- Hacking methodologies, technologies and tools change. Consequently, a vulnerability fixed today does not mean that it has been fixed forever. It is possible that a vulnerability fixed today - with a patch or re-configuration - might still be exploited in future. For this reason we recommend to periodically run Quixxi Scan.
- Vulnerability tests are not capable of detecting any inherent hardware-based or firmware-based problems, performances problems or functionality problems.
- The generated report is the only output/outcome from Scan provided by Quixxi to the customers.
- Quixxi automated vulnerability assessment is a static scan that works best when provided with a developer build of an APK or an IPA which is not obfuscated or encrypted. Quixxi will still be able to process obfuscated or encrypted files but the results of such scans might be partial and may not be complete. For best results, kindly upload files that are not obfuscated or encrypted.

Sample Report

Codeless App Security

Quixxi Shield protects your app against hackers looking to clone, tamper or exploit your app. NO CODING REQUIRED!

TRY QUIXXI SHIELD

Sample Report

TESTING METHODOLOGY

Quixxi tests the app performing a static analysis inspired to the OWASP MASVS standard. The severity and the categorization of the different vulnerabilities are evaluated by Quixxi depending on the technical and business impacts they can have on the app, its publisher and its end users. The assessment involve vulnerabilities concerning the following areas:

Client attacks

Network attacks

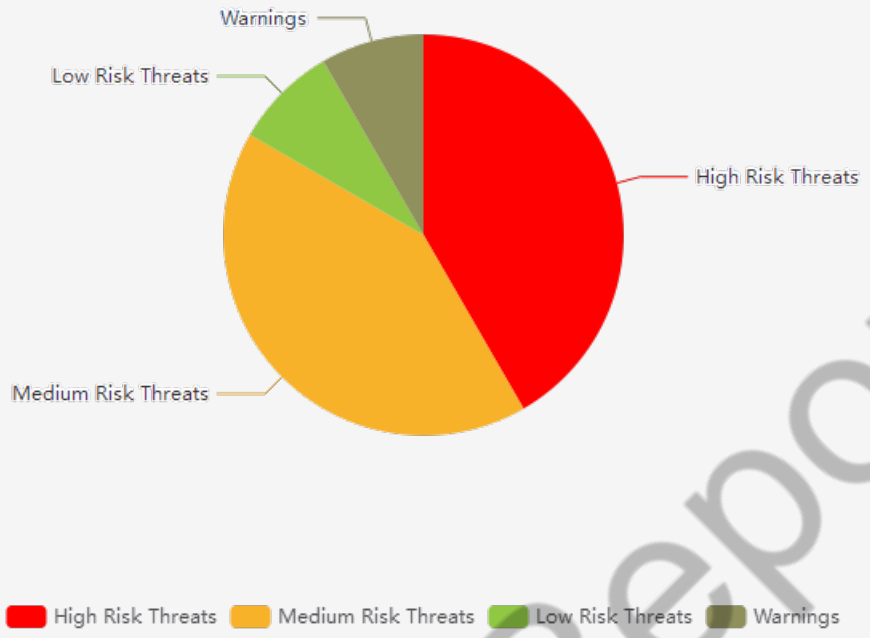
ABBREVIATIONS

CWE – Common Weakness Enumeration	CVE – Common Vulnerabilities and Exposures
MASVS – Mobile Application Security Verification Standard	OWASP – Open Web Application Security Project

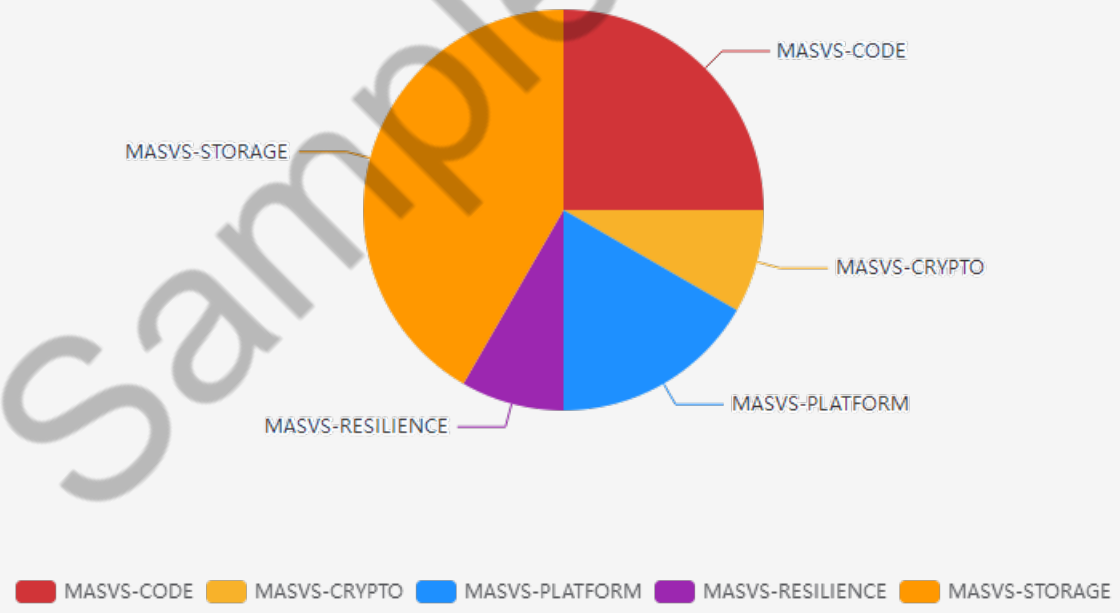
Total Risk Threats	High Risk Threats	Medium Risk Threats	Low Risk Threats	Warnings
12	5	5	1	1

Sample Report

Risk Threats Analysis



Security Requirements



Sample Report

MASVS-CODE: Security best practices for data processing and keeping the app up-to-date	3 / 5 Vulnerabilities detected
MASVS-CRYPTO: Cryptographic functionality used to protect sensitive data	1 / 8 Vulnerabilities detected
MASVS-NETWORK: Secure network communication between the mobile app and remote endpoints (data-in-transit)	0 Vulnerabilities detected
MASVS-PLATFORM: Secure interaction with the underlying mobile platform and other installed apps	2 / 11 Vulnerabilities detected
MASVS-RESILIENCE: Resilience to reverse engineering and tampering attempts	1 / 4 Vulnerabilities detected
MASVS-STORAGE: Secure storage of sensitive data on a device (data-at-rest)	5 / 9 Vulnerabilities detected

The above classification of vulnerabilities is based on OWASP MASVS standards

Sample Report

FILE INFORMATION

File Name: dynamic_analysis_test_app5.apk

Size: 16.34 MB

MD5: c0ae429452ed15862a497676d41590e0

SHA1: 58d7bca9d4c9d07b31bc9f634eb8eb015aaeabf1

SHA256: 0cf900506cbdfe80196daa6a2573af0a45795ceb27143541732c34443f147d71

APP INFORMATION

App Name: Dynamic Analysis Test App

Package Name: com.quixxi.dynamic.testapp

App Version Name: 1.0

App Version Code: 1

Main Activity: com.quixxi.dynamic.testapp.ui.login.LoginActivity

Target SDK: 30

Min SDK: 21

Platform: Android

Permissions:

android.permission.FOREGROUND_SERVICE
android.permission.READ_EXTERNAL_STORAGE
android.permission.ACCESS_NETWORK_STATE
android.permission.ACCESS_MEDIA_LOCATION
android.permission.WRITE_EXTERNAL_STORAGE
android.permission.INTERNET
android.permission.MANAGE_EXTERNAL_STORAGE
android.permission.REORDER_TASKS

Sample Report

VULNERABILITY SCAN - FAILED SCENARIOS

Severity
High

VULNERABILITY

Improper Export of your Android Activities

OWASP MASVS	Common Weakness Enumeration
6.2 L2	CWE-926
Known Exploits	**Common Vulnerability Scoring System**
CVE-2017-12816	CVE-2017-12816-CVSS 3.0 Score 9.8

Best Practices:

https://github.com/OWASP/owasp-mstg/blob/master/Document/0x05h-Testing-Platform-Interaction.md#activity-permission-enforcement

Reference URL[s]:

https://stackoverflow.com/a/11551693
https://developer.android.com/guide/topics/manifest/activity-element.html#prmsn
https://developer.android.com/guide/topics/permissions/defining
https://developer.android.com/guide/topics/manifest/permission-element
https://cwe.mitre.org/data/definitions/926.html
https://developer.android.com/training/articles/security-tips

CVSS BaseScore and Vector

CVE-2017-12816
Version & Base Score : CVSS 3.0 Score 9.8
CVSS Scoring Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

THREAT

An Android Activity provides the window in which the app draws its UI. This window typically fills the screen, but may be smaller than the screen and float on top of other windows. Generally, one Activity implements one screen in an app

Most apps contain multiple screens, which means they comprise multiple Activities. Typically one of the activities in the app is specified as the Main Activity, which is the first screen to appear when the user launches the app

Developers can switch from one Activity to another using Components known as Intents that can also pass in additional values to the subsequent screen. However if there are not enough security measures an attacker might launch directly the second Activity bypassing the first one

Sample Report

RISK

Vulnerable Activities could be used to bypass security restrictions in an app [e.g. authentication or pass code screens, navigating to unauthorized locations of the app]. So by exploiting this technique a malicious application can gain access to any sensitive information.

FIX

There are a few solutions that help mitigate the issue: if you don't want the Activity to be launched by components of other applications please delete any Intent Filters because their declaration is enough to turn that Activity into exported by default do not assume the input is from a trusted source and validate each time to avoid your app to crash if you need to restrict the access to your Activity to specific callers then use an appropriate 'android:permission'. For example, consider using the signature 'android:protectionLevel' on permissions for IPC communication between applications provided by a single developer. Since Android 5.0 and superior, when naming a custom <permission> please bear in mind that the system does not permit the user to install other packages with the same permission name, unless those packages are signed with the same certificate as the first package. In order to avoid naming collisions it is recommended to use reverse-domain-style naming for custom permissions, for example com.example.myapp.MY_PERMISSION

The following Activity is not protected. An intent-filter exists.

```
 <activity android:theme="@android:style/Theme" android:name="androidx.test.core.app.InstrumentationActivityInvoke
r$BootstrapActivity">

        <intent-filter>

          <action android:name="android.intent.action.MAIN"/>

        </intent-filter>

      </activity>

 <activity android:theme="@android:style/Theme" android:name="androidx.test.core.app.InstrumentationActivityInvoke
r$EmptyActivity">

        <intent-filter>

          <action android:name="android.intent.action.MAIN"/>

        </intent-filter>

      </activity>

 <activity android:theme="@android:style/Theme.Dialog" android:name="androidx.test.core.app.InstrumentationActivity
Invoker$EmptyFloatingActivity">

        <intent-filter>

          <action android:name="android.intent.action.MAIN"/>

        </intent-filter>

      </activity>
```

Sample Report

Severity
High

VULNERABILITY

Improper Export of your Android Broadcast Receiver

OWASP MASVS	Common Weakness Enumeration
6.2 L2	CWE-927
	CWE-925
Known Exploits	**Common Vulnerability Scoring System**
CAPEC-499	
CAPEC-501	

Reference URL[s]:

https://developer.android.com/guide/topics/manifest/receiver-element.html#exported
http://blog.palominolabs.com/2013/05/13/android-security/index.html
https://stackoverflow.com/a/38376435
https://developer.android.com/guide/topics/permissions/defining
https://developer.android.com/guide/topics/manifest/permission-element
https://cwe.mitre.org/data/definitions/926.html
https://developer.android.com/training/articles/security-tips

CVSS BaseScore and Vector

CAPEC-499

CAPEC-501

THREAT

Broadcast Receivers enable applications to receive Intents that are broadcast by the system or by other applications, even when other Components of the application are not running. Many Android applications use them to receive Intents without properly verifying if coming from an authorized source

Explicit Intents explicitly name the class of the target Android Component that will handle the Intent. Implicit Intents are used without a class name and in this case Android will help determine an appropriate Component to handle the Intent

Sample Report

RISK

A major problem with respect to intent spoofing is that registering an Intent Filter to receive implicit Intents makes that Component exported by default. This default "exported" behavior allows all the applications to send both implicit and explicit Intents to that Component

And for an exported=true Component if the Intents are explicit then they don't even have to match the Intent Filter. So - ironically - creating an Intent Filter for a Component greatly widens the scope of Intents that Android will allow to be sent to it. Developers must not rely on Intent Filters for security

FIX

There are a few solutions that help mitigate the issue: if you don't want the Broadcast Receiver to receive messages from sources outside its application please delete any Intent Filters because their declaration is enough to turn that Broadcast Receiver into exported by default, even if you explicitly declared it as android:exported=false the current Android behavior is that a Broadcast Receiver without Intent Filters is not exported but if you want you can also explicit this adding android:export="false" do not assume the input is from a trusted source and validate each time to avoid your app to crash if you need to restrict the access to your Broadcast Receiver to specific callers then use an appropriate 'android:permission'. For example, consider using the signature 'android:protectionLevel' on permissions for IPC communication between applications provided by a single developer. Since Android 5.0 and superior, when naming a custom <permission> please bear in mind that the system does not permit the user to install other packages with the same permission name, unless those packages are signed with the same certificate as the first package. To avoid naming collisions, it is recommended to use reverse-domain-style naming for custom permissions, for example com.example.myapp.MY_PERMISSION

The following Broadcast Receiver is not protected and export flag is set(android:exported=true).

```
 <receiver android:name="com.quixxi.dynamic.testapp.receivers.FirstReceiver" android:enabled="true" android:export
ed="true">

        <intent-filter>

           <action android:name="com.quixxi.dynamic.testapp.android.action.broadcast"/>

        </intent-filter>

    </receiver>
```

Sample Report

Severity
High

VULNERABILITY

Unsafe files deletion

OWASP MASVS	Common Weakness Enumeration
2.10	CWE-200
Known Exploits	**Common Vulnerability Scoring System**
CVE-2018-3987	CVE-2018-3987-CVSS 3.0 Score 5.5

Best Practices:

https://www.youtube.com/watch?v=tGw1fxUD-uY

Reference URL[s]:

https://stackoverflow.com/a/22372329

CVSS BaseScore and Vector

CVE-2018-3987
Version & Base Score : CVSS 3.0 Score 5.5
CVSS Scoring Vector : CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

THREAT

When you delete a file using file.delete(), only the reference to the file is removed from the file system table. The file still exists on disk until other data overwrites it, leaving it vulnerable to recovery

RISK

Because of what just explained everything you delete may be recovered by any user or attacker, especially on rooted devices

FIX

First of all the developer must consider if there is an absolute need to save sensitive data in files that will be kept on customer side

If this is really required then - whenever there will be the need to remove a file - it is strongly suggested to first empty it [e.g. writing an empty string and saving it] before calling the actual deletion. This will fix the vulnerability

Additionally, please consider to encrypt any sensitive file in such a way that its content is protected against snooping also during its known life on device, not just after deletion

We have detected '.delete()' in the file com/getkeepsafe/relinker/ReLinkerInstance.java

Sample Report

```
line : 6 import com.getkeepsafe.relinker.elf.ElfParser;

line : 7 import java.io.File;

line : 8 import java.io.FilenameFilter;

line : 158              if (this.force || !file.getAbsolutePath().equals(workaroundFile.getAbsolutePath())) {

line : 159                  file.delete();

line : 160              }
```

We have detected '.delete()' in the file io/realm/internal/Util.java

```
line : 8 import io.realm.log.RealmLog;

line : 9 import java.io.File;

line : 10 import java.io.PrintWriter;

line : 66               File file = files[i2];

line : 67               boolean deleteResult = file.delete();

line : 68               if (!deleteResult) {

line : 77       }

line : 78       if (managementFolder.exists() && !managementFolder.delete()) {

line : 79           RealmLog.warn(String.format(Locale.ENGLISH, "Realm temporary folder at %s cannot be deleted", ma
nagementFolder.getAbsolutePath()), new Object[0]);

line : 81       if (realmFile.exists()) {

line : 82           realmDeleted = realmFile.delete();

line : 83           if (!realmDeleted) {

line : 88       }

line : 89       if (fifoFile.exists() && !fifoFile.delete()) {

line : 90           RealmLog.warn(String.format(Locale.ENGLISH, ".note file at %s cannot be deleted", fifoFile.getAbsolut
ePath()), new Object[0]);
```

Sample Report

Severity
High

VULNERABILITY

App seeks Root/Super user privileges

OWASP MASVS	Common Weakness Enumeration
8.1 R	CWE-250
Known Exploits	**Common Vulnerability Scoring System**
CVE-2019-16273	CVE-2019-16273-CVSS 3.0 Score 9.8

Reference URL[s]:

https://mobile-security.gitbook.io/mobile-security-testing-guide/android-testing-guide/0x05j-testing-resiliency-against-reverse-engineering#effectiveness-assessment

CVSS BaseScore and Vector

CVE-2019-16273
Version & Base Score : CVSS 3.0 Score 9.8
CVSS Scoring Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

THREAT

This App may request root (Super User) privileges.

RISK

An app can request root access and it only works if the device allows it or the phone has been rooted. Once an app has root privileges, it essentially has complete administrative control over the device on which the app has been installed which carries serious security implications.

FIX

Using one or more of the below techniques, determine whether app detects and requests for root access: -Multiple detection methods are scattered throughout the app (as opposed to putting everything into a single method). -The root detection mechanisms operate on multiple API layers (Java APIs, native library functions, assembler/system calls). -The mechanisms are somehow original (they are not copied and pasted from StackOverflow or other sources).

We have detected 'com.noshufou.android.su' in the file com/scottyab/rootbeer/Const.java

Sample Report

line : 8 static final String BINARY_SU = "su";

line : 9 static final String[] knownRootAppsPackages = {"com.noshufou.android.su", "com.noshufou.android.su.elite", "eu.chainfire.supersu", "com.koushikdutta.superuser", "com.thirdparty.superuser", "com.yellowes.su", "com.topjohnwu.magisk", "com.kingroot.kinguser", "com.kingo.root", "com.smedialink.oneclickroot", "com.zhiqupk.root.global", "com.alephzain.framaroot"};

line : 10 public static final String[] knownDangerousAppsPackages = {"com.koushikdutta.rommanager", "com.koushikdutta.rommanager.license", "com.dimonvideo.luckypatcher", "com.chelpus.lackypatch", "com.ramdroid.appquarantine", "com.ramdroid.appquarantinepro", "com.android.vending.billing.InAppBillingService.COIN", "com.android.vending.billing.InAppBillingService.LUCK", "com.chelpus.luckypatcher", "com.blackmartalpha", "org.blackmart.market", "com.allinone.free", "com.repodroid.app", "org.creeplays.hack", "com.baseappfull.fwd", "com.zmapp", "com.dv.marketmod.installer", "org.mobilism.android", "com.android.wp.net.log", "com.android.camera.update", "cc.madkite.freedom", "com.solohsu.android.edxp.manager", "org.meowcat.edxposed.manager", "com.xmodgame", "com.cih.game_cih", "com.charles.lpoqasert", "catch_.me_.if_.you_.can_"};

Severity
High

VULNERABILITY

Cleartext Storage of Sensitive Information in app source code

OWASP MASVS	**Common Weakness Enumeration**
2.14 L2	CWE-312
Known Exploits	**Common Vulnerability Scoring System**
CVE-2018-19981	CVE-2018-19981-CVSS 3.0 Score 7.2

Best Practices:

https://aws-amplify.github.io/aws-sdk-android/docs/reference/com/amazonaws/auth/CognitoCachingCredentialsProvider.html

CVSS BaseScore and Vector

CVE-2018-19981
Version & Base Score : CVSS 3.0 Score 7.2
CVSS Scoring Vector : CVSS:3.0/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H

THREAT

App source may contain hardcoded sensitive information such as Oauth tokens, API keys, passwords any known intellectual property that maybe considered sensitive.

Sample Report

RISK

Sometimes mobile application developers leave sensitive data in the mobile app source code. They are not the companys crown jewels per se, but they are potential clues for malicious hackers to find them. It is a mistake to hardcode security components, such as security tokens or encryption keys, or privileged bits of code, such as API keys or proprietary algorithms, on the mobile device. Doing so may give malicious hackers the opportunity to steal those secrets by reverse-engineering the mobile app

FIX

If sensitive data is still required to be stored locally, it should be encrypted using a key derived from hardware backed storage which requires authentication

We have detected '_key = "rx2.purge-period-seconds"' in the file io/reactivex/internal/schedulers/SchedulerPoolFactory.java

line : 17 public static final boolean purge_enabled;

line : 18 static final string purge_enabled_key = "rx2.purge-enabled";

line : 19 public static final int purge_period_seconds;

line : 20 static final string purge_period_seconds_key = "rx2.purge-period-seconds";

line : 21 static final atomicreference<scheduledexecutorservice> purge_thread = new atomicreference<>();

Severity
Medium

VULNERABILITY

ADB Backup allowed

OWASP MASVS	Common Weakness Enumeration
2.8 L2	CWE-530
	CWE-312
Known Exploits	**Common Vulnerability Scoring System**
CVE-2017-16835	CVE-2017-16835-CVSS 3.0 Score 7.5

Best Practices:

https://resources.infosecinstitute.com/android-hacking-security-part-15-hacking-android-apps-using-backup-techniques/
https://github.com/OWASP/owasp-mstg/blob/1.1.3/Document/0x05d-Testing-Data-Storage.md#testing-backups-for-sensitive-data-mstg-storage-8
https://securitygrind.com/exploiting-android-backup/

Sample Report

Reference URL[s]:

https://androidquest.wordpress.com/2014/09/18/backup-applications-on-android-phone-with-adb/
https://developer.android.com/guide/topics/data/autobackup#IncludingFiles

CVSS BaseScore and Vector

CVE-2017-16835
Version & Base Score : CVSS 3.0 Score 7.5
CVSS Scoring Vector : CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

THREAT

The Android operating system offers a backup/restore mechanism of installed packages through the ADB utility. The full backup of applications including the private files stored on data partition is allowed by default

RISK

The attacker can access the backup and any sensitive data included [e.g. passwords] is exposed

FIX

In order to fix this issue please consider to disable ADB Backup for any apps that might include sensitive content. You can also customize your backup by implementing a BackupAgent class

ADB Backup is enabled for this app (default: ENABLED)

```
<application android:theme="@style/AppTheme" android:label="@string/app_name" android:icon="@mipmap/ic_launcher" android:debuggable="true" android:allowBackup="true" android:installLocation="preferExternal" android:supportsRtl="true" android:networkSecurityConfig="@xml/network_security_config" android:roundIcon="@mipmap/ic_launcher" android:appComponentFactory="androidx.core.app.CoreComponentFactory">
```

Severity
Medium

VULNERABILITY

Debuggable App

Sample Report

OWASP MASVS	Common Weakness Enumeration
7.2 L2	CWE-215

Known Exploits	Common Vulnerability Scoring System
CVE-2019-16273	CVE-2019-16273-CVSS 3.0 Score 9.8
CVE-2019-16272	CVE-2019-16272-CVSS 3.0 Score 9.8
CVE-2019-16241	CVE-2019-16241-CVSS 3.0 Score 6.8
CVE-2017-3750	CVE-2017-3750-CVSS 3.0 Score 6.4

Reference URL[s]:

https://labs.mwrinfosecurity.com/blog/debuggable-apps-in-android-market/
https://developer.android.com/guide/topics/manifest/application-element.html#debug
https://stackoverflow.com/questions/43363767/how-can-i-set-androiddebuggable-false
https://tekeye.uk/android/examples/android-debug-vs-release-build
https://security.stackexchange.com/a/74265
https://stackoverflow.com/questions/17580199/sending-a-sms-on-android-through-adb

CVSS BaseScore and Vector

CVE-2019-16273
Version & Base Score : CVSS 3.0 Score 9.8
CVSS Scoring Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CVE-2019-16272
Version & Base Score : CVSS 3.0 Score 9.8
CVSS Scoring Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CVE-2019-16241
Version & Base Score : CVSS 3.0 Score 6.8
CVSS Scoring Vector : CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CVE-2017-3750
Version & Base Score : CVSS 3.0 Score 6.4
CVSS Scoring Vector : CVSS:3.0/AV:P/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H

Sample Report

THREAT

The android debug bridge [ADB] is used to connect the developer's system to an Android phone or to the emulator. The adbd daemon can be started after enabling the USB debugging and connecting the phone to the computer. ADB can now be used on the computer to spawn a shell, forward ports and do other debugging related tasks. It can also be used to get a list of all running debuggable applications and to connect to one of these applications for debugging purposes

Despite being turned off by default, the debug mode can be enabled during the development of an app setting android:debuggable="true" in the AndroidManifest.xml file. If this attribute is set, the application will try to connect to a local UNIX socket "@jdwp-control". This socket is opened on the device by adbd [ADB server daemon process], waiting for debuggable applications to register by connecting to the socket. ADB running on the computer can now request a connection to any of them and adbd will forward a connection between the debugger and the debuggee. Any further communication will now take place using the standard Java Debug Wire Protocol [JDWP]

Debuggable applications will try to connect to the UNIX socket “@jdwp-control”, even if adbd is not running. This is the case on default configurations of Android [USB debugging disabled] or on phones in development mode unless they are connected to a computer

RISK

Any application can open this UNIX socket and thus trick any debuggable application into connecting to it. Therefore, shipping a debuggable application means that anyone with physical access to the device can execute arbitrary code under that debuggable application's permission

If the application holds sensitive data, it will be fairly straightforward to extract them from the application. The risk impact of this vulnerability is only limited by the imagination of a hacker. For instance, it can be used to: spy on the user's communication send premium rate SMS without the user’s knowledge make background calls take pictures forward emails etc. etc. Built-in system apps that come along with the phone have system-level privileges - i.e. higher than the other ones which are installed from Play Store - so they might be subject to more attacks

Also, be aware that the physical access to any rooted device is enough to get the access to the apps secrets stored in the device, independently from the selection made for the debuggable flag

FIX

Quixxi App Shield can automatically disable the debuggable flag when forgotten, without the need to select any specific Shield entry. Dragging'n'dropping your app will be enough

Otherwise, please disable the debuggable mode from the release version. In order to avoid the confusion between test and production versions please edit the "buildTypes" section of the "build.gradle(Module:app)" file so to distinguish the debuggable behaviour between the two versions of the app

DEBUG mode is ON(android:debuggable="true") in AndroidManifest.xml

```
<application android:theme="@style/AppTheme" android:label="@string/app_name" android:icon="@mipmap/ic_launcher" android:debuggable="true" android:allowBackup="true" android:installLocation="preferExternal" android:supportsRtl="true" android:networkSecurityConfig="@xml/network_security_config" android:roundIcon="@mipmap/ic_launcher" android:appComponentFactory="androidx.core.app.CoreComponentFactory">
```

Sample Report

Severity
Medium

VULNERABILITY

Debugging Information Provision

OWASP MASVS	Common Weakness Enumeration
7.4 L2	CWE-215
Known Exploits	**Common Vulnerability Scoring System**
CAPEC-133	CVE-2018-6599-CVSS 3.0 Score 5.5
CVE-2018-6599	

Reference URL[s]:

https://github.com/b66l/OASAM/blob/master/oasam-leak-information-leak/oasam-leak-002-information-leak-to-log-files.md

CVSS BaseScore and Vector

CAPEC-133

CVE-2018-6599
Version & Base Score : CVSS 3.0 Score 5.5
CVSS Scoring Vector : CVSS:3.0/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

THREAT

Applications can output runtime information using the android.util.Log class. Logcats are usually collected [e.g. via adb logcat -v long > logcat.txt] in order to bugfix the app but they should be removed once done

RISK

If the app is configured to check the behavior in code sections where the developer is dealing with users sensitive data then this info can leak. Besides this, any log line can theoretically give unrequired details about the app's logic

For example an app log might reveal a strategy for detecting rooted phones based on two different methods. This will tell a hacker that in order to run the app on rooted phones he will need to overcome two checks

Sample Report

FIX

Quixxi App Shield can automatically remove the debug logs [Log.i(), Log.d(), Log.v(), System.out.println()] preventing them to provide information by enabling the "Remove app logs" Shield entry

One common manual solution is declaring and using a custom log class, so that the output log is automatically turned on/off based on Debug/Release. If the app includes any third-party libraries it is developer's responsibility to guarantee safety conditions also for the additional files included

We have detected 'Log.v(' in the file com/scottyab/rootbeer/util/QLog.java

```
line : 2
line : 3 import android.util.Log;
line : 4 import java.io.PrintWriter;
line : 19 if (isELoggable()) {
line : 20 Log.e(TAG, getTrace() + String.valueOf(obj));
line : 21 Log.e(TAG, getThrowableTrace(cause));
line : 22 Log.e(TAG_GENERAL_OUTPUT, getTrace() + String.valueOf(obj));
line : 23 Log.e(TAG_GENERAL_OUTPUT, getThrowableTrace(cause));
line : 24 }
line : 28 if (isELoggable()) {
line : 29 Log.e(TAG, getTrace() + String.valueOf(obj));
line : 30 Log.e(TAG_GENERAL_OUTPUT, getTrace() + String.valueOf(obj));
line : 31 }
line : 41 if (isWLoggable()) {
line : 42 Log.w(TAG, getTrace() + String.valueOf(obj));
line : 43 Log.w(TAG, getThrowableTrace(cause));
line : 44 Log.w(TAG_GENERAL_OUTPUT, getTrace() + String.valueOf(obj));
line : 45 Log.w(TAG_GENERAL_OUTPUT, getThrowableTrace(cause));
line : 46 }
line : 50 if (isWLoggable()) {
line : 51 Log.w(TAG, getTrace() + String.valueOf(obj));
line : 52 Log.w(TAG_GENERAL_OUTPUT, getTrace() + String.valueOf(obj));
line : 53 }
line : 57 if (isILoggable()) {
line : 58 Log.i(TAG, getTrace() + String.valueOf(obj));
line : 59 }
line : 63 if (isDLoggable()) {
line : 64 Log.d(TAG, getTrace() + String.valueOf(obj));
line : 65 }
line : 69 if (isVLoggable()) {
line : 70 Log.v(TAG, getTrace() + String.valueOf(obj));
line : 71 }
```

We have detected 'System.out.print' in the file io/realm/internal/RealmCore.java

```
line : 38 if (jnilib != null) {
line : 39 System.out.println("!!! Realm debug version loaded. !!!\n");
line : 40 } else {
```

We have detected 'System.out.print' in the file kotlinx/coroutines/debug/AgentPremain.java

```
line : 97 } else {
line : 98 System.out.println((Object) "Cannot perform coroutines dump, debug probes are disabled");
line : 99 }
```

Sample Report

Severity
Medium

VULNERABILITY

Missing protection against screenshots & screensharing

OWASP MASVS	Common Weakness Enumeration
2.7 L2	CWE-200
Known Exploits	**Common Vulnerability Scoring System**
CVE-2015-6630	CVE-2015-6630-CVSS 2.0 Score 4.3

Best Practices:

https://www.securityweek.com/screenaudio-capture-vulnerability-impacts-lions-share-android-devices

Reference URL[s]:

https://www.xda-developers.com/android-screen-recording-vulnerability/
https://threatpost.com/mobile-malware-captures-keystrokes-screengrabs/103973/

CVSS BaseScore and Vector

CVE-2015-6630
Version & Base Score : CVSS 2.0 Score 4.3
CVSS Scoring Vector : AV:N/AC:M/Au:N/C:P/I:N/A:N

THREAT

Screenshots and screensharing are useful and powerful features whenever they can provide an additional value to the target app

RISK

In specific cases though, not protecting the app against screenshot captures and screensharing exposes the user to the leakage of sensitive information [e.g. healthcare apps]

FIX

Quixxi App Shield can automatically prevent against screenshots and screensharing functionalities by enabling the "Disable screenshots capture & screen sharing" Shield option. The same option will automatically provide protection to blur the app preview when it is placed in the background

Sample Report

Severity
Medium

VULNERABILITY

No blurring for the app in background

OWASP MASVS	Common Weakness Enumeration
2.9 L2	CWE-200
Known Exploits	**Common Vulnerability Scoring System**
CVE-2015-6630	CVE-2015-6630-CVSS 2.0 Score 4.3

Best Practices:

https://github.com/OWASP/owasp-mstg/blob/1.1.3/Document/0x05d-Testing-Data-Storage.md#finding-sensitive-information-in-auto-generated-screenshots-mstg-storage-9

CVSS BaseScore and Vector

CVE-2015-6630
Version & Base Score : CVSS 2.0 Score 4.3
CVSS Scoring Vector : AV:N/AC:M/Au:N/C:P/I:N/A:N

THREAT

When a user puts an app in background, the same information shown in the foreground is displayed in the background as preview

RISK

The lack of background screen blurring can - as a minimum - cause data leakage [e.g. banking amounts]. Such previews can be also potentially exported via screenshots

FIX

Quixxi App Shield can automatically blur the app preview when it is put in background by enabling the "Disable screenshots capture & screen sharing" Shield option. The same option will automatically provide protection against screenshots and screen sharing

Sample Report

Severity
Low

VULNERABILITY

Missing check for the download source

OWASP MASVS	Common Weakness Enumeration
7.1	CWE-610
Known Exploits	**Common Vulnerability Scoring System**
CVE-2018-9582	CVE-2018-9582-CVSS 3.0 Score 7.8

Best Practices:

http://www.securityfocus.com/bid/1064
https://source.android.com/security/bulletin/2019-01-01.html

Reference URL[s]:

https://developer.android.com/reference/android/content/pm/PackageManager.html#getInstallerPackageName%28java.lang.String%29

CVSS BaseScore and Vector

CVE-2018-9582
Version & Base Score : CVSS 3.0 Score 7.8
CVSS Scoring Vector : CVSS:3.0/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

THREAT

The same app can be downloaded from different markets and the installer verification code is used to understand where the app was downloaded from [e.g. Google Play, Amazon, Samsung, black markets, etc]

RISK

If the installer verification code doesn't match a trusted app store [e.g. Google Play, Amazon, Samsung] we cannot ascertain the authenticity of the app that will run on the phone

This exposes the end-users to be able to run tampered versions of your original app. Therefore, your brand can be used as a Trojan to inject malwares

Sample Report

FIX

Quixxi App Shield will automatically remove the issue by enabling the "Terminate the app if installation from Unknown Sources" together with "Send events and device info to Quixxi portal" Shield entries. The app will be able to run only if installed from Google Play, Amazon or Samsung store

Severity
Warning

VULNERABILITY

Weak Java Hash Code implementation

OWASP MASVS	**Common Weakness Enumeration**
3.4 [L1, L2]	CWE-327
Known Exploits	**Common Vulnerability Scoring System**
-	-

Best Practices:

https://security.stackexchange.com/questions/52877/how-secure-is-javas-hashcode

Reference URL[s]:

https://developer.android.com/reference/java/lang/Object#hashCode()

CVSS BaseScore and Vector

-

THREAT

This App uses Java Hash Code. It is a weak hash function and should never be used in Secure Crypto Implementation. public int hashCode () Returns a hash code value for the object. This method is supported for the benefit of hash tables such as those provided by HashMap.

Sample Report

RISK

Java Virtual Machine's Object.hashCode() method has atleast two known issues: (1) The integer does not remain consistent from one execution of an application to another execution of the same application. (2) It implements a hash table that is open to collision and DoS (Denial of Service) attacks - You see, the output of hashCode() (in its default implementation) is a 32-bit unsigned integer, that is roughly 4 billion unique hashCodes. Sounds quite a lot? Well, not so much. Applying the birthday problem in this case shows as that with about 77000 objects, you have about 50% chance of collision. 50% chance of two objects having the the same hashCode.

FIX

If the developer is to insist on using hashes as object identifiers, then it is much better to have your own method instead of hashCode() to use for your key identifiers (for example, getMySpecialHashKey(). You can uses something like MessageDigest.getInstance("SHA-256") to digest the object into a nice 256-bit key.

We have detected '.hashCode()' in the file io/reactivex/Notification.java

```
line : 53 if (o != null) {
line : 54 return o.hashCode();
line : 55 }
```

We have detected '.hashCode()' in the file io/reactivex/internal/util/OpenHashSet.java

```
line : 30 int m = this.mask;
line : 31 int pos = mix(value.hashCode()) & m;
line : 32 T curr2 = a[pos];
line : 57 int m = this.mask;
line : 58 int pos = mix(value.hashCode()) & m;
line : 59 T curr2 = a[pos];
line : 86 }
line : 87 int slot = mix(curr.hashCode()) & m;
line : 88 if (pos > pos) {
line : 113 } while (a[i] == null);
line : 114 int pos2 = mix(a[i].hashCode()) & m;
line : 115 if (b[pos2] != null) {
```

We have detected '.hashCode()' in the file io/reactivex/internal/util/NotificationLite.java

```
line : 26 public int hashCode() {
line : 27 return this.e.hashCode();
line : 28 }
```

We have detected '.hashCode()' in the file io/reactivex/internal/functions/ObjectHelper.java

```
line : 29 if (o != null) {
line : 30 return o.hashCode();
line : 31 }
```

We have detected '.hashCode()' in the file io/reactivex/schedulers/Timed.java

Sample Report

```
line : 42 T t = this.value;
line : 43 int h = t != null ? t.hashCode() : 0;
line : 44 long j = this.time;
line : 45 return (((h * 31) + ((int) (j ^ (j >>> 31)))) * 31) + this.unit.hashCode();
line : 46 }
```

We have detected '.hashCode()' in the file io/realm/com_quixxi_dynamic_testapp_data_model_PersonRealmProxy.java

```
line : 547 int i = 0;
line : 548 int result = (17 * 31) + (realmName != null ? realmName.hashCode() : 0);
line : 549 int result2 = result * 31;
line : 550 if (tableName != null) {
line : 551 i = tableName.hashCode();
line : 552 }
```

We have detected '.hashCode()' in the file io/realm/ProxyState.java

```
line : 47 public int hashCode() {
line : 48 return this.listener.hashCode();
line : 49 }
```

We have detected '.hashCode()' in the file io/realm/RealmConfiguration.java

```
line : 234 int i = 0;
line : 235 int result = file != null ? file.hashCode() : 0;
line : 236 int i2 = result * 31;
line : 237 String str = this.realmFileName;
line : 238 int result2 = i2 + (str != null ? str.hashCode() : 0);
line : 239 int result3 = ((result2 * 31) + this.canonicalPath.hashCode()) * 31;
line : 240 String str2 = this.assetFilePath;
line : 241 int hashCode = str2 != null ? str2.hashCode() : 0;
line : 242 long j = this.schemaVersion;
line : 244 RealmMigration realmMigration = this.migration;
line : 245 int result5 = (((((((result4 + (realmMigration != null ? realmMigration.hashCode() : 0)) * 31) + (this.deleteReal
mIfMigrationNeeded ? 1 : 0)) * 31) + this.durability.hashCode()) * 31) + this.schemaMediator.hashCode()) * 31;
line : 246 RxObservableFactory rxObservableFactory = this.rxObservableFactory;
line : 247 int result6 = (result5 + (rxObservableFactory != null ? rxObservableFactory.hashCode() : 0)) * 31;
line : 248 Realm.Transaction transaction = this.initialDataTransaction;
line : 249 int result7 = (((result6 + (transaction != null ? transaction.hashCode() : 0)) * 31) + (this.readOnly ? 1 : 0)) * 3
1;
line : 250 CompactOnLaunchCallback compactOnLaunchCallback = this.compactOnLaunch;
line : 251 if (compactOnLaunchCallback != null) {
line : 252 i = compactOnLaunchCallback.hashCode();
line : 253 }
```

We have detected '.hashCode()' in the file io/realm/MutableRealmInteger.java

```
line : 202 }
line : 203 return thisValue.hashCode();
line : 204 }
```

We have detected '.hashCode()' in the file io/realm/DynamicRealmObject.java

```
line : 681 int i = 0;
line : 682 int result = (17 * 31) + (realmName != null ? realmName.hashCode() : 0);
line : 683 int result2 = result * 31;
line : 684 if (tableName != null) {
line : 685 i = tableName.hashCode();
line : 686 }
```

Sample Report

We have detected '.hashCode()' in the file io/realm/internal/ObserverPairList.java

```
line : 41 int i = 0;
line : 42 int result = (17 * 31) + (observer != null ? observer.hashCode() : 0);
line : 43 int result2 = result * 31;
line : 45 if (s != null) {
line : 46 i = s.hashCode();
line : 47 }
```

We have detected '.hashCode()' in the file io/realm/internal/ObservableCollection.java

```
line : 48 public int hashCode() {
line : 49 return this.listener.hashCode();
line : 50 }
```

We have detected '.hashCode()' in the file io/realm/internal/RealmProxyMediator.java

```
line : 64 public int hashCode() {
line : 65 return getModelClasses().hashCode();
line : 66 }
```

We have detected '.hashCode()' in the file io/realm/internal/util/Pair.java

```
line : 26 int i = 0;
line : 27 int hashCode = f == null ? 0 : f.hashCode();
line : 28 S s = this.second;
line : 29 if (s != null) {
line : 30 i = s.hashCode();
line : 31 }
```

We have detected '.hashCode()' in the file io/realm/rx/ObjectChange.java

```
line : 40 public int hashCode() {
line : 41 int result = this.object.hashCode();
line : 42 int i = result * 31;
line : 43 ObjectChangeSet objectChangeSet = this.changeset;
line : 44 int result2 = i + (objectChangeSet != null ? objectChangeSet.hashCode() : 0);
line : 45 return result2;
```

We have detected '.hashCode()' in the file io/realm/rx/CollectionChange.java

```
line : 40 public int hashCode() {
line : 41 int result = this.collection.hashCode();
line : 42 int i = result * 31;
line : 43 OrderedCollectionChangeSet orderedCollectionChangeSet = this.changeset;
line : 44 int result2 = i + (orderedCollectionChangeSet != null ? orderedCollectionChangeSet.hashCode() : 0);
line : 45 return result2;
```

We have detected '.hashCode()' in the file kotlinx/coroutines/DebugKt.java

```
line : 43 if (r0 == 0) goto L_0x006f
line : 44 int r4 = r0.hashCode()
line : 45 if (r4 == 0) goto L_0x0043
```

We have detected '.hashCode()' in the file kotlinx/coroutines/CompletedWithCancellation.java

```
line : 50 Object obj = this.result;
line : 51 return ((obj == null ? 0 : obj.hashCode()) * 31) + this.onCancellation.hashCode();
line : 52 }
```

We have detected '.hashCode()' in the file kotlinx/coroutines/CompletedContinuation.java

Sample Report

```
line : 75 int i = 0;
line : 76 int hashCode = (obj == null ? 0 : obj.hashCode()) * 31;
line : 77 CancelHandler cancelHandler = this.cancelHandler;
line : 78 int hashCode2 = (hashCode + (cancelHandler == null ? 0 : cancelHandler.hashCode())) * 31;
line : 79 Function1<Throwable, Unit> function1 = this.onCancellation;
line : 80 int hashCode3 = (hashCode2 + (function1 == null ? 0 : function1.hashCode())) * 31;
line : 81 Object obj2 = this.idempotentResume;
line : 82 int hashCode4 = (hashCode3 + (obj2 == null ? 0 : obj2.hashCode())) * 31;
line : 83 Throwable th = this.cancelCause;
line : 84 if (th != null) {
line : 85 i = th.hashCode();
line : 86 }
```

We have detected '.hashCode()' in the file kotlinx/coroutines/CoroutineName.java

```
line : 38 public int hashCode() {
line : 39 return this.name.hashCode();
line : 40 }
```

We have detected '.hashCode()' in the file kotlinx/coroutines/JobCancellationException.java

```
line : 51 Intrinsics.checkNotNull(message);
line : 52 int hashCode = ((message.hashCode() * 31) + this.job.hashCode()) * 31;
line : 53 Throwable cause = getCause();
line : 54 return hashCode + (cause == null ? 0 : cause.hashCode());
line : 55 }
```

We have detected '.hashCode()' in the file kotlinx/coroutines/internal/SegmentOrClosed.java

```
line : 42 }
line : 43 return obj.hashCode();
line : 44 }
```

We have detected '.hashCode()' in the file kotlinx/coroutines/internal/InlineList.java

```
line : 44 }
line : 45 return obj.hashCode();
line : 46 }
```

We have detected '.hashCode()' in the file kotlinx/coroutines/internal/ThreadLocalKey.java

```
line : 36 public int hashCode() {
line : 37 return this.threadLocal.hashCode();
line : 38 }
```

We have detected '.hashCode()' in the file kotlinx/coroutines/channels/ChannelResult.java

```
line : 44 }
line : 45 return obj.hashCode();
line : 46 }
line : 135 if (th != null) {
line : 136 return th.hashCode();
line : 137 }
```

We have detected '.hashCode()' in the file kotlinx/coroutines/debug/internal/HashedWeakRef.java

```
line : 14 super(t, referenceQueue);
line : 15 this.hash = t != null ? t.hashCode() : 0;
line : 16 }
```

Sample Report

We have detected '.hashCode()' in the file kotlinx/coroutines/debug/internal/ConcurrentWeakMap.java

```
line : 177 public final V getImpl(K k) {
line : 178 int index = index(k.hashCode());
line : 179 while (true) {
```

We have detected '.hashCode()' in the file org/bson/BsonDecimal128.java

```
line : 37 public int hashCode() {
line : 38 return this.value.hashCode();
line : 39 }
```

We have detected '.hashCode()' in the file org/bson/BsonObjectId.java

```
line : 47 public int hashCode() {
line : 48 return this.value.hashCode();
line : 49 }
```

We have detected '.hashCode()' in the file org/bson/BsonRegularExpression.java

```
line : 46 public int hashCode() {
line : 47 int result = this.pattern.hashCode();
line : 48 return (result * 31) + this.options.hashCode();
line : 49 }
```

We have detected '.hashCode()' in the file org/bson/Document.java

```
line : 259 public int hashCode() {
line : 260 return this.documentAsMap.hashCode();
line : 261 }
```

We have detected '.hashCode()' in the file org/bson/BsonElement.java

```
line : 38 int i = 0;
line : 39 int result = getName() != null ? getName().hashCode() : 0;
line : 40 int i2 = result * 31;
line : 41 if (getValue() != null) {
line : 42 i = getValue().hashCode();
line : 43 }
```

We have detected '.hashCode()' in the file org/bson/BsonJavaScript.java

```
line : 33 public int hashCode() {
line : 34 return this.code.hashCode();
line : 35 }
```

We have detected '.hashCode()' in the file org/bson/BsonString.java

```
line : 41 public int hashCode() {
line : 42 return this.value.hashCode();
line : 43 }
```

We have detected '.hashCode()' in the file org/bson/BsonSymbol.java

```
line : 37 public int hashCode() {
line : 38 return this.symbol.hashCode();
line : 39 }
```

We have detected '.hashCode()' in the file org/bson/BsonDbPointer.java

Sample Report

```
line : 47 public int hashCode() {
line : 48 int result = this.namespace.hashCode();
line : 49 return (result * 31) + this.id.hashCode();
line : 50 }
```

We have detected '.hashCode()' in the file org/bson/BsonDocument.java

```
line : 419 public int hashCode() {
line : 420 return entrySet().hashCode();
line : 421 }
```

We have detected '.hashCode()' in the file org/bson/BsonJavaScriptWithScope.java

```
line : 45 public int hashCode() {
line : 46 int result = this.code.hashCode();
line : 47 return (result * 31) + this.scope.hashCode();
line : 48 }
```

We have detected '.hashCode()' in the file org/bson/BsonDocumentWrapper.java

```
line : 115 public int hashCode() {
line : 116 return getUnwrapped().hashCode();
line : 117 }
```

We have detected '.hashCode()' in the file org/bson/BsonArray.java

```
line : 172 public int hashCode() {
line : 173 return this.values.hashCode();
line : 174 }
```

We have detected '.hashCode()' in the file org/bson/RawBsonArray.java

```
line : 99 public int hashCode() {
line : 100 return super.hashCode();
line : 101 }
```

We have detected '.hashCode()' in the file org/bson/RawBsonDocument.java

```
line : 257 public int hashCode() {
line : 258 return toBsonDocument().hashCode();
line : 259 }
```

We have detected '.hashCode()' in the file org/bson/codecs/BsonTypeClassMap.java

```
line : 83 public int hashCode() {
line : 84 return this.map.hashCode();
line : 85 }
```

We have detected '.hashCode()' in the file org/bson/codecs/IterableCodecProvider.java

```
line : 55 public int hashCode() {
line : 56 int result = this.bsonTypeClassMap.hashCode();
line : 57 int i = result * 31;
line : 58 Transformer transformer = this.valueTransformer;
line : 59 int result2 = i + (transformer != null ? transformer.hashCode() : 0);
line : 60 return result2;
```

We have detected '.hashCode()' in the file org/bson/codecs/MapCodecProvider.java

Sample Report

```
line : 56 public int hashCode() {
line : 57 int result = this.bsonTypeClassMap.hashCode();
line : 58 int i = result * 31;
line : 59 Transformer transformer = this.valueTransformer;
line : 60 int result2 = i + (transformer != null ? transformer.hashCode() : 0);
line : 61 return result2;
```

We have detected '.hashCode()' in the file org/bson/codecs/DocumentCodecProvider.java

```
line : 60 public int hashCode() {
line : 61 int result = this.bsonTypeClassMap.hashCode();
line : 62 int i = result * 31;
line : 63 Transformer transformer = this.valueTransformer;
line : 64 int result2 = i + (transformer != null ? transformer.hashCode() : 0);
line : 65 return result2;
```

We have detected '.hashCode()' in the file org/bson/codecs/pojo/TypeParameterMap.java

```
line : 72 public int hashCode() {
line : 73 return getPropertyToClassParamIndexMap().hashCode();
line : 74 }
```

We have detected '.hashCode()' in the file org/bson/codecs/pojo/TypeData.java

```
line : 146 public int hashCode() {
line : 147 int result = getType().hashCode();
line : 148 return (result * 31) + getTypeParameters().hashCode();
line : 149 }
```

We have detected '.hashCode()' in the file org/bson/codecs/pojo/ClassModel.java

```
line : 125 public int hashCode() {
line : 126 int result = getType().hashCode();
line : 127 int i = 0;
line : 128 int result2 = (((((((result * 31) + getInstanceCreatorFactory().hashCode()) * 31) + (this.discriminatorEnabled
? 1 : 0)) * 31) + (getDiscriminatorKey() != null ? getDiscriminatorKey().hashCode() : 0)) * 31) + (getDiscriminator() != n
ull ? getDiscriminator().hashCode() : 0)) * 31;
line : 129 if (getIdPropertyModelHolder() != null) {
line : 130 i = getIdPropertyModelHolder().hashCode();
line : 131 }
line : 132 return ((((result2 + i) * 31) + getPropertyModels().hashCode()) * 31) + getPropertyNameToTypeParameterM
ap().hashCode();
line : 133 }
```

We have detected '.hashCode()' in the file org/bson/codecs/pojo/PropertyModel.java

```
line : 122 int i = 0;
line : 123 int result = getName() != null ? getName().hashCode() : 0;
line : 124 int result2 = ((((((((((result * 31) + (getReadName() != null ? getReadName().hashCode() : 0)) * 31) + (getWri
teName() != null ? getWriteName().hashCode() : 0)) * 31) + (getTypeData() != null ? getTypeData().hashCode() : 0)) *
31) + (getCodec() != null ? getCodec().hashCode() : 0)) * 31) + (getPropertySerialization() != null ? getPropertySerializ
ation().hashCode() : 0)) * 31;
line : 125 Boolean bool = this.useDiscriminator;
line : 126 int result3 = (((((result2 + (bool != null ? bool.hashCode() : 0)) * 31) + (getPropertyAccessor() != null ? getPr
opertyAccessor().hashCode() : 0)) * 31) + (getError() != null ? getError().hashCode() : 0)) * 31;
line : 127 if (getCachedCodec() != null) {
line : 128 i = getCachedCodec().hashCode();
line : 129 }
```

We have detected '.hashCode()' in the file org/bson/codecs/pojo/IdPropertyModelHolder.java

Sample Report

```
line : 59 int i = 0;
line : 60 int result = propertyModel != null ? propertyModel.hashCode() : 0;
line : 61 int i2 = result * 31;
line : 63 if (idGenerator != null) {
line : 64 i = idGenerator.hashCode();
line : 65 }
```

We have detected '.hashCode()' in the file org/bson/internal/ChildCodecRegistry.java

```
line : 67 ChildCodecRegistry<?> childCodecRegistry = this.parent;
line : 68 int result = childCodecRegistry != null ? childCodecRegistry.hashCode() : 0;
line : 69 return (((result * 31) + this.registry.hashCode()) * 31) + this.codecClass.hashCode();
line : 70 }
```

We have detected '.hashCode()' in the file org/bson/internal/OverridableUuidRepresentationCodecRegistry.java

```
line : 56 public int hashCode() {
line : 57 int result = this.wrapped.hashCode();
line : 58 return (result * 31) + this.uuidRepresentation.hashCode();
line : 59 }
```

We have detected '.hashCode()' in the file org/bson/internal/ProvidersCodecRegistry.java

```
line : 69 public int hashCode() {
line : 70 return this.codecProviders.hashCode();
line : 71 }
```

We have detected '.hashCode()' in the file org/bson/util/AbstractCopyOnWriteMap.java

```
line : 202 public final int hashCode() {
line : 203 return this.delegate.hashCode();
line : 204 }
line : 476 public int hashCode() {
line : 477 return getDelegate().hashCode();
line : 478 }
```

We have detected '.hashCode()' in the file org/bson/util/ComputingMap.java

```
line : 123 public int hashCode() {
line : 124 return this.map.hashCode();
line : 125 }
```

We have detected '.hashCode()' in the file org/bson/types/Code.java

```
line : 31 public int hashCode() {
line : 32 return this.code.hashCode();
line : 33 }
```

We have detected '.hashCode()' in the file org/bson/types/Symbol.java

```
line : 31 public int hashCode() {
line : 32 return this.symbol.hashCode();
line : 33 }
```

We have detected '.hashCode()' in the file org/bson/types/CodeWScope.java

```
line : 28 public int hashCode() {
line : 29 return getCode().hashCode() ^ this.scope.hashCode();
line : 30 }
```

Sample Report

We have detected '.hashCode()' in the file org/bson/types/CodeWithScope.java

```
line : 35 public int hashCode() {
line : 36 return getCode().hashCode() ^ this.scope.hashCode();
line : 37 }
```

We have detected '.hashCode()' in the file retrofit2/Utils.java

```
line : 354 public int hashCode() {
line : 355 int hashCode = Arrays.hashCode(this.typeArguments) ^ this.rawType.hashCode();
line : 356 Type type = this.ownerType;
line : 357 return hashCode ^ (type != null ? type.hashCode() : 0);
line : 358 }
line : 396 public int hashCode() {
line : 397 return this.componentType.hashCode();
line : 398 }
line : 452 Type type = this.lowerBound;
line : 453 return (type != null ? type.hashCode() + 31 : 1) ^ (this.upperBound.hashCode() + 31);
line : 454 }
```


Codeless App Security

Quixxi Shield protects your app against hackers looking to clone, tamper or exploit your app. NO CODING REQUIRED!

TRY QUIXXI SHIELD

Sample Report